```
list (2 => $third, 3 => $fourth) = $myArr1;
[$first, $second] = $myArr1;
list ('lang' => $lang, 'ver' => $ver) = $myArr2;

var_dump($first);
var_dump($second);
var_dump($third);
var_dump($fourth);

print("{$lang}{$ver} FTW!\n");

Output:
string(5) "Alice"
string(4) "Rita"
string(3) "Sue"
string(3) "Bob"
PHP8.4 FTW!
```

In the first assignment (using `list`), we asked for explicit indexes (2 and 3), and the values of those third and fourth elements are assigned to the variables in the `list` parentheses. The second assignment statement uses square brackets and is syntactically equivalent to `list ($first, $second) = $myArr;`. It is, of course, also possible to use explicit indexes in the square bracket form: `[2 => $third, 3 => $fourth] = $myArr` is perfectly legal syntax. If using keys, all values must have them; it isn't possible to mix the two forms: `list($first, 1 => $second)` is illegal syntax.

With `extract()`

It is possible to save some effort when extracting scalars from arrays. Quite often the newly created variable names will align with the keys of the array providing the data, in which case the built-in function `extract()` will deal with creating the variables for you. If an array `['foo' => 'bar']` is passed to `extract()`, the variable `$foo` is created: it has a value of `"bar"` and exists only in the current scope (if `extract()` is called inside your function, the new variables exist only within that function). Bear in mind that `$0`, `$1`, etc., would be illegal variable names, so extracting from an indexed array requires a little extra work: